برنامه نویسی به زبان سی

پردازش فایل

دکتر مهران صفایانی

safayani@iut.ac.ir

safayani.iut.ac.ir

https://www.aparat.com/mehran.safayani

https://github.com/safayani/Programming_Basics_course

دانشکده مهندسی برق و کامپیوتر، دانشگاه صنعتی اصفهان، اصفهان، ایران

File Processing in C

- **Files** enable **long-term data retention** after a program ends.
- Data stored on secondary storage (SSDs, flash drives, hard drives).
- Each file is a **sequential stream of bytes**.

FILE Structure and File Pointers

- **FILE Structure (**<stdio.h>**)**
  - fopen() returns a **pointer to a** FILE **structure**.
  - Contains system info for processing the file (e.g., file descriptor, file control block).
- **File Pointers**
  - stdin, stdout– pointers to standard streams.
  - Each open file has its own FILE pointer.

Common File-Processing Functions

- **Character I/O**
  - fgetc(filePtr) – Reads one character (like getchar()).
  - fputc(char, filePtr) – Writes one character (like putchar()).
- **String and Formatted I/O**
  - fgets() / fputs() – Read/write a line of text.
  - fscanf() / fprintf() – Formatted input/output to files.
  - fread() / fwrite() – For binary data (discussed later).

Creating a Sequential-Access File

- **Overview**
  - C imposes **no structure** on files; you define the record format.
  - Example: Accounts receivable system (account number, name, balance).
- **Record key**: Account number.
- **Steps**
  - Define a FILE pointer.
  - Open the file with fopen().
  - Write data using fprintf().
  - Close the file with fclose().

Opening and Closing Files

- fopen() **– Opening a File**

```
FILE *cfPtr = fopen("clients.txt", "w");
```

- **Arguments**: Filename (with optional path) and file-open mode.
- **Mode** "w": Opens for writing. Creates file if nonexistent; **overwrites** if existing.
- Returns NULL if file cannot be opened.
- fclose() **– Closing a File**

```
fclose(cfPtr);
```

- Frees system resources.
- Always close files when no longer needed.

Checking for End-of-File (EOF)

- feof() **Function**
- Checks if end-of-file indicator is set for a stream.
- Example:

```
while (!feof(stdin)) { ... }
```

- **EOF key combinations**:
  - **Windows**: Ctrl + Z then Enter
  - **macOS/Linux**: Ctrl + D

Writing to a File with fprintf()

- fprintf() **– Formatted File Output**
- Similar to printf(), but includes a FILE pointer argument.
- Example:

```
fprintf(cfPtr, "%d %s %.2f\n", account, name, balance);
```

- Can also write to standard output using stdout.

Text Modes

Mode	Name	Read	Write	Append	File Exists	File Doesn't Exist	Position After Open	Exclusive
"r"	Read	✅ Yes	❌ No	❌ No	Opens file	Error (returns NULL)	Beginning	❌ No
"w"	Write	❌ No	✅ Yes	❌ No	Truncates to zero	Creates new	Beginning	❌ No
"a"	Append	❌ No	✅ Yes	✅ Yes	Preserves content	Creates new	End (writing) Beginning (reading with seek)	❌ No
"r+"	Read/Write	✅ Yes	✅ Yes	❌ No	Opens file	Error (returns NULL)	Beginning	❌ No
"w+"	Write/Read	✅ Yes	✅ Yes	❌ No	Truncates to zero	Creates new	Beginning	❌ No
"a+"	Read/Append	✅ Yes	✅ Yes	✅ Yes	Preserves content	Creates new	End (writing) Beginning (reading with seek)	❌ No
"wx"	Exclusive Write	❌ No	✅ Yes	❌ No	Error (returns NULL)	Creates new	Beginning	✅ Yes
"w+x"	Exclusive Write/Read	✅ Yes	✅ Yes	❌ No	Error (returns NULL)	Creates new	Beginning	✅ Yes

Common File-Processing Errors

- **Typical Errors**
  - Opening a **nonexistent file** for reading.
  - **Insufficient permissions** to access the file.
  - **Insufficient disk space** for writing.
  - Opening in "w" mode **accidentally overwriting** an existing file.
- **Best Practices**
  - **Always check** if fopen() returns NULL.
  - **Close files** explicitly when done.
  - Use **exclusive mode (**"wx"**)** to prevent accidental overwrites.
  - Test file operations in a controlled environment.

```c
#include <stdio.h>
int main(void) {
   FILE *cfPtr = fopen("clients.txt", "w");
   if (cfPtr == NULL) {       puts("File could not be opened");     return 1;     }
   puts("Enter the account, name, and balance.");
   puts("Enter EOF (Ctrl+Z then Enter) to end input.");
   printf("? ");
   int account;
   char name[30];
   double balance;
   while (scanf("%d %29s %lf", &account, name, &balance) == 3) {
      fprintf(cfPtr, "%d %s %.2f\n", account, name, balance);
      printf("? ");    }
   fclose(cfPtr);
   return 0;}
```

Reading Data from Sequential-Access Files

- *// Open file with minimum required permissions*

FILE *file = fopen("data.txt", "r"); *// Read-only mode*

- **Read-only mode ("r")**: Prevents accidental modifications
- **Security best practice**: Only grant necessary access
- **Error prevention**: Cannot overwrite data accidentally
- **Reading vs Writing Mode**
- **Writing**: "w", "a", "r+" - Can modify
- **Reading**: "r" - View only, safe access

Reading with fscanf()

- *// Syntax: Similar to scanf()*
- int fscanf(FILE *stream, const char *format, ...);
- *// Example from Fig. 11.2*
- FILE *cfPtr = fopen("clients.txt", "r");
- fscanf(cfPtr, "%d%s%lf", &account, name, &balance);
- **Key Characteristics**
- **First argument**: FILE pointer (not stdin)
- **Format identical** to scanf()
- **Returns number** of successfully read items
- **Returns EOF** on end-of-file or error

```c
// fig11_02.c
// Reading and printing a sequential file
#include <stdio.h>

int main(void) {
   FILE *cfPtr = NULL; // cfPtr = clients.txt file pointer

   // fopen opens file; exits program if file cannot be opened
   if ((cfPtr = fopen("clients.txt", "r")) == NULL) {
      puts("File could not be opened");
   }
   else { // read account, name and balance from file
      int account = 0; // account number
      char name[30] = ""; // account name
      double balance = 0.0; // account balance

      printf("%-10s%-13s%s\n", "Account", "Name", "Balance");
      fscanf(cfPtr, "%d%29s%lf", &account, name, &balance);

      // while not end of file
      while (!feof(cfPtr)) {
         printf("%-10d%-13s%7.2f\n", account, name, balance);
         fscanf(cfPtr, "%d%29s%lf", &account, name, &balance);
      }

      fclose(cfPtr); // fclose closes the file
   }
}
```

```
Account   Name          Balance
100       Jones           24.98
200       Doe            345.67
300       White            0.00
400       Stone          -42.16
500       Rich           224.62
```

rewind() Function

- *// Reposition to beginning of file*

rewind(cfPtr);

- **Use Cases**
- Process file multiple times
- Start reading from beginning after EOF
- Reset for re-scanning operations

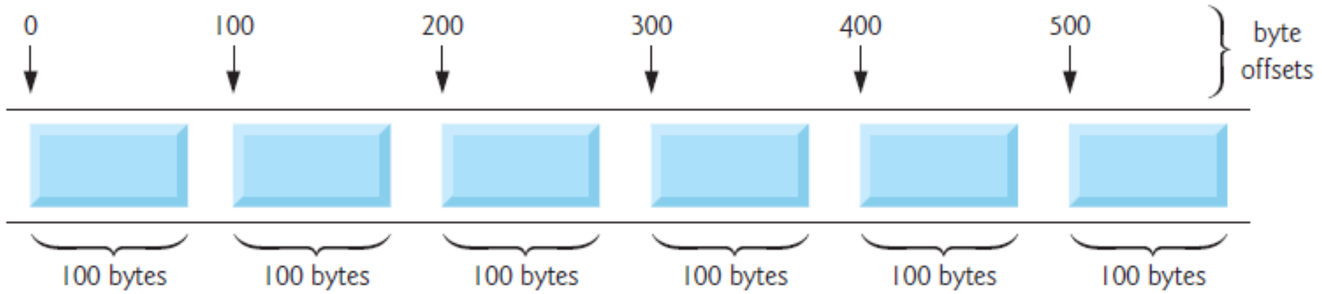
```c
#include <stdio.h>
int main() {
    FILE *f = fopen("demo.txt", "r+");
       // Write some numbers
    for (int i = 1; i <= 5; i++) {
       fprintf(f, "%d ", i);    }
     rewind(f);  // Back to start  Read them back
    int num;
    printf("Numbers: ");
    while (fscanf(f, "%d", &num) == 1) {
       printf("%d ", num);
    }
     fclose(f);
    return 0;
}
```

```
Reading file first time:
Line 1
Line 2
Line 3

Reading file second time (after rewind):
Line 1
Line 2
Line 3
```

Random-Access Files

- Fixed-length records for direct access
- Transaction processing applications
- Binary file operations with fread/fwrite
- Record Location = Record Number × Record Size
- **Example:**
  - Record size = 100 bytes
  - Record #5 location = 5 × 100 = byte 500
  - Can jump directly to byte 500!
- **Benefits**
  - **Immediate access** to any record
  - **No searching** through other records
  - **Safe updates** without affecting other data
  - **Efficient** for large files



Binary File Operations

- **fwrite() Function**

// Writes binary data to file

size_t fwrite(const void *ptr, size_t size, size_t count, FILE *stream);

- **fread() Function**

// Reads binary data from file

size_t fread(void *ptr, size_t size, size_t count, FILE *stream);

- **Arguments**
  - ptr - Pointer to data/memory location
  - size - Size of each element (bytes)
  - count - Number of elements
  - stream - FILE pointer

Text vs Binary Output

- **Text Output (fprintf)**

```
int number = 1234567890;
fprintf(fPtr, "%d", number);
// Could write 10 characters = ~10 bytes
// Human-readable: "1234567890"
```

- **Binary Output (fwrite)**

```
int number = 1234567890;
fwrite(&number, sizeof(int), 1, fPtr);
// Always writes sizeof(int) bytes (typically 4)
// System-dependent binary format
```

Binary Modes

Mode	Name	Read	Write	Append	File Exists	File Doesn't Exist	Position After Open	Exclusive
"rb"	Read Binary	✅ Yes	❌ No	❌ No	Opens file	Error (returns NULL)	Beginning	❌ No
"wb"	Write Binary	❌ No	✅ Yes	❌ No	Truncates to zero	Creates new	Beginning	❌ No
"ab"	Append Binary	❌ No	✅ Yes	✅ Yes	Preserves content	Creates new	End (writing) Beginning (reading with seek)	❌ No
"rb+" / "r+b"	Read/Write Binary	✅ Yes	✅ Yes	❌ No	Opens file	Error (returns NULL)	Beginning	❌ No
"wb+" / "w+b"	Write/Read Binary	✅ Yes	✅ Yes	❌ No	Truncates to zero	Creates new	Beginning	❌ No
"ab+" / "a+b"	Read/Append Binary	✅ Yes	✅ Yes	✅ Yes	Preserves content	Creates new	End (writing) Beginning (reading with seek)	❌ No
"wbx"	Exclusive Write Binary	❌ No	✅ Yes	❌ No	Error (returns NULL)	Creates new	Beginning	✅ Yes
"wb+x" / "w+bx"	Exclusive Write/Read Binary	✅ Yes	✅ Yes	❌ No	Error (returns NULL)	Creates new	Beginning	✅ Yes

Example Data Structure

- Create transaction system with:
  - **100 fixed-length records**
  - **Account number** (record key), **Last name**, **First name, Balance**
  - Operations: Update, Insert, Delete, List
- **Struct Definition**

```
struct clientData {
    unsigned int accountNum;  // 4 bytes
    char lastName[15];        // 15 bytes
    char firstName[10];       // 10 bytes
    double balance;           // 8 bytes (typically)
};  // Total: ~37 bytes fixed length
```

-

```c
// fig11_04.c
// Creating a random-access file sequentially
#include <stdio.h>

// clientData structure definition
struct clientData {
   int account;
   char lastName[15];
   char firstName[10];
   double balance;
};

int main(void) {
   FILE *cfPtr = NULL; // accounts.dat file pointer

   // fopen opens the file; exits if file cannot be opened
   if ((cfPtr = fopen("accounts.dat", "wb")) == NULL) {
      puts("File could not be opened.");
   }
   else {
      // create clientData with default information
      struct clientData blankClient = {0, "", "", 0.0};

      // output 100 blank records to file
      for (int i = 1; i <= 100; ++i) {
         fwrite(&blankClient, sizeof(struct clientData), 1, cfPtr);
      }

      fclose (cfPtr); // fclose closes the file
   }
}
```

What is fseek()?

- Changes the file position indicator (File Pointer)
- Enables random access to files
- Move forward/backward within a file
- Jump to specific positions

```
int fseek(FILE *stream, long offset, int whence);
```

- stream: Pointer to FILE object
- offset: Number of bytes to offset
- whence: Starting position for the offset
- **Return Value**
  - **0** on success
  - **Non-zero** on failure (e.g., seeking before file start)

The whence Parameter

Value	Standard Constant	Description	Example
0	SEEK_SET	From **beginning** of file	fseek(fp, 100, SEEK_SET)
1	SEEK_CUR	From **current** position	fseek(fp, -10, SEEK_CUR)
2	SEEK_END	From **end** of file	fseek(fp, -50, SEEK_END)

Example:

```
fseek(cfPtr, offset, SEEK_SET);  // Position pointer
fwrite(&client, sizeof(struct clientData), 1, cfPtr);  // Write
```

Positioning with fseek()

- **Position Calculation Formula**

```
// For account numbers 1-100:
offset = (client.account - 1) * sizeof(struct clientData);
fseek(cfPtr, offset, SEEK_SET);
```

- **Why (account - 1)?**File Byte Positions:
  - Byte 0: Record #1 (Account 1)
  - Byte 100: Record #2 (Account 2)
  - Byte 200: Record #3 (Account 3)
- Calculation: (3 - 1) × 100 = Byte 200 ✓

```c
int main(void) {
   FILE *cfPtr = NULL; // accounts.dat file pointer

   // fopen opens the file; exits if file cannot be opened
   if ((cfPtr = fopen("accounts.dat", "rb+")) == NULL) {
      puts("File could not be opened.");
   }
   else {
      // create clientData with default information
      struct clientData client = {0, "", "", 0.0};

      // require user to specify account number
      printf("%s", "Enter account number (1 to 100, 0 to end input): ");
      scanf("%d", &client.account);

      // user enters information, which is copied into file
      while (client.account != 0) {
         // user enters last name, first name and balance
         printf("%s", "Enter lastname, firstname, balance: ");

         // set record lastName, firstName and balance value
         fscanf(stdin, "%14s%9s%lf", client.lastName,
            client.firstName, &client.balance);

         // seek position in file to user-specified record
         fseek(cfPtr, (client.account - 1) *
            sizeof(struct clientData), SEEK_SET);

         // write user-specified information in file
         fwrite(&client, sizeof(struct clientData), 1, cfPtr);

         // enable user to input another account number
         printf("%s", "\nEnter account number: ");
         scanf("%d", &client.account);
      }

      fclose(cfPtr); // fclose closes the file
   }
}
```

```
Enter account number (1 to 100, 0 to end input): 37
Enter lastname, firstname, balance: Barker Doug 0.00

Enter account number: 29
Enter lastname, firstname, balance: Brown Nancy -24.54
```

The fread() Function

- **Function Prototype**

size_t fread(void *ptr, size_t size, size_t count, FILE *stream);

- **Reading a Single Record**
- struct clientData client;
- fread(&client, sizeof(struct clientData), 1, cfPtr);
- **What Happens**
  - Reads sizeof(struct clientData) bytes
  - From current file position pointer location
  - Stores data in client variable
  - Returns number of items successfully read

```
int main(void) {
FILE *cfPtr = fopen("accounts.dat", "rb");
if (cfPtr == NULL) {      puts("File could not be opened.");      return 1;    }
printf("%-6s%-16s%-11s%10s\n", "Acct", "Last Name","First Name", "Balance");
struct clientData client;
while (fread(&client, sizeof(struct clientData), 1, cfPtr) == 1) {
     // fread returns 1 only if it successfully read a complete record
     if (client.account != 0) {  // Skip empty records
       printf("%-6d%-16s%-11s%10.2f\n", client.account,
            client.lastName, client.firstName, client.balance);   }
  }
if (feof(cfPtr)) {
     // Normal end of file - all good
   } else if (ferror(cfPtr)) {      printf("Error occurred while reading file.\n");   }
    fclose(cfPtr);
   return 0;}
```

```
Acct  Last Name       First Name    Balance
29    Brown           Nancy          -24.54
33    Dunn            Stacey         314.33
37    Barker          Doug             0.00
88    Smith           Dave           258.34
96    Stone           Sam             34.98
```

```c
#include <stdio.h>
int main() {
    // Create a file with exact size records
    FILE *f = fopen("test.dat", "wb");
    int data[3] = {100, 200, 300};
    fwrite(data, sizeof(int), 3, f);
    fclose(f);
    f = fopen("test.dat", "rb");
    int value;    int count = 0;
    printf("Using WRONG pattern (feof before fread):\n");
    while (!feof(f)) {
        size_t result = fread(&value, sizeof(int), 1, f);
        printf("feof()=%d, fread() returned %zu, value=%d\n",feof(f), result, value);
        count++;    }
    fclose(f);
    printf("Read attempts: %d (should be 3, but got %d!)\n", 3, count);
    return 0;}
```

What is the problem with this code?

```
Using WRONG pattern (feof before fread):
feof()=0, fread() returned 1, value=100
feof()=0, fread() returned 1, value=200
feof()=0, fread() returned 1, value=300
feof()=1, fread() returned 0, value=300
Read attempts: 3 (should be 3, but got 4!)
```

```c
int main(void){
   FILE *cfPtr = NULL; // accounts.dat file pointer

   // fopen opens the file; exits if file cannot be opened
   if ((cfPtr = fopen("accounts.dat", "rb")) == NULL) {
      puts("File could not be opened.");
   }
   else {
      printf("%-6s%-16s%-11s%10s\n", "Acct", "Last Name",
         "First Name", "Balance");

      // read all records from file (until eof)
      while (!feof(cfPtr)) {
         // read a record
         struct clientData client = {0, "", "", 0.0};
         size_t result =
            fread(&client, sizeof(struct clientData), 1, cfPtr);

         // display record
         if (result != 0 && client.account != 0) {
            printf("%-6d%-16s%-11s%10.2f\n", client.account,
               client.lastName, client.firstName, client.balance);
         }
      }

      fclose(cfPtr); // fclose closes the file
   }
}
```

```
Acct  Last Name       First Name    Balance
29    Brown           Nancy          -24.54
33    Dunn            Stacey         314.33
37    Barker          Doug             0.00
88    Smith           Dave           258.34
96    Stone           Sam             34.98
```

Case Study - Transaction Processing System

- **Program Capabilities**
- **Create formatted account list** (Option 1)
- **Update existing accounts** (Option 2)
- **Add new accounts** (Option 3)
- **Delete accounts** (Option 4)
- **Exit program** (Option 5)

```
5  #include <stdio.h>
6
7  // clientData structure definition
8  struct clientData {
9     int account;
10    char lastName[15];
11    char firstName[10];
12    double balance;
13 };

14
15  // prototypes
16  int enterChoice(void);
17  void textFile(FILE *readPtr);
18  void updateRecord(FILE *fPtr);
19  void newRecord(FILE *fPtr);
20  void deleteRecord(FILE *fPtr);
21
```

```c
int main(void) {
   FILE *cfPtr = NULL; // accounts.dat file pointer

   // fopen opens the file; exits if file cannot be opened
   if ((cfPtr = fopen("accounts.dat", "rb+")) == NULL) {
      puts("File could not be opened.");
   }
   else {
      int choice = 0; // user

      // enable user to specify action
      while ((choice = enterChoice()) != 5) {
         switch (choice) {
            case 1: // create text file from record file
               textFile(cfPtr);
               break;
            case 2: // update record
               updateRecord(cfPtr);
               break;
            case 3: // create record
               newRecord(cfPtr);
               break;
            case 4: // delete existing record
               deleteRecord(cfPtr);
               break;
            default: // display message for invalid choice
               puts("Incorrect choice");
               break;
         }
      }

      fclose(cfPtr); // fclose closes the file
   }
}
```

```c
// enable user to input menu choice
int enterChoice(void) {
   // display available options
   printf("%s", "\nEnter your choice\n"
      "1 - store a formatted text file of accounts called\n"
      "    \"accounts.txt\" for printing\n"
      "2 - update an account\n"
      "3 - add a new account\n"
      "4 - delete an account\n"
      "5 - end program\n? ");

   int menuChoice = 0; // variable to store user
   scanf("%d", &menuChoice); // receive choice from user
   return menuChoice;
}
```

```c
// create formatted text file for printing
void textFile(FILE *readPtr) {
   FILE *writePtr = NULL; // accounts.txt file pointer

   // fopen opens the file; exits if file cannot be opened
   if ((writePtr = fopen("accounts.txt", "w")) == NULL) {
      puts("File could not be opened.");
   }

   else {
      rewind(readPtr); // sets pointer to beginning of file
      fprintf(writePtr, "%-6s%-16s%-11s%10s\n",
         "Acct", "Last Name", "First Name","Balance");

      // copy all records from random-access file into text file
      while (!feof(readPtr)) {
         // create clientData with default information
         struct clientData client = {0, "", "", 0.0};
         size_t result =
            fread(&client, sizeof(struct clientData), 1, readPtr);

         // write single record to text file
         if (result != 0 && client.account != 0) {
            fprintf(writePtr, "%-6d%-16s%-11s%10.2f\n", client.account,
               client.lastName, client.firstName, client.balance);
         }
      }

      fclose(writePtr); // fclose closes the file
   }
}
```

```
127 // create and insert record
128 void newRecord(FILE *fPtr) {
129    // obtain number of account to create
130    printf("%s", "Enter new account number (1 - 100): ");
131    int account = 0; // account number
132    scanf("%d", &account);
133
134    // move file pointer to correct record in file
135    fseek(fPtr, (account - 1) * sizeof(struct clientData), SEEK_SET);
136
137    // read record from file
138    struct clientData client = {0, "", "", 0.0};
139    fread(&client, sizeof(struct clientData), 1, fPtr);
140
141    // display error if account already exists
142    if (client.account != 0) {
143       printf("Account #%d already contains information.\n",
144          client.account);
145    }
146    else { // create record
147       // user enters last name, first name and balance
148       printf("%s", "Enter lastname, firstname, balance\n? ");
149       scanf("%14s%9s%lf", &client.lastName, &client.firstName,
150          &client.balance);
151
152       client.account = account;
153
154       // move file pointer to correct record in file
155       fseek(fPtr, (client.account - 1) * sizeof(struct clientData),
156          SEEK_SET);
157
158       // insert record in file
159       fwrite(&client, sizeof(struct clientData), 1, fPtr);
160    }
161 }
```

```c
// delete an existing record
void deleteRecord(FILE *fPtr) {
   // obtain number of account to delete
   printf("%s", "Enter account number to delete (1 - 100): ");
   int account = 0; // account number
   scanf("%d", &account);

   // move file pointer to correct record in file
   fseek(fPtr, (account - 1) * sizeof(struct clientData), SEEK_SET);

   // read record from file
   struct clientData client = {0, "", "", 0.0};
   fread(&client, sizeof(struct clientData), 1, fPtr);

   // display error if record does not exist
   if (client.account == 0) {
      printf("Account %d does not exist.\n", account);
   }
   else { // delete record
      // move file pointer to correct record in file
      fseek(fPtr, (account - 1) * sizeof(struct clientData), SEEK_SET);

      struct clientData blankClient = {0, "", "", 0}; // blank client

      // replace existing record with blank record
      fwrite(&blankClient, sizeof(struct clientData), 1, fPtr);
   }
}
```

```c
// update balance in record
void updateRecord(FILE *fPtr) {
   // obtain number of account to update
   printf("%s", "Enter account to update (1 - 100): ");
   int account = 0; // account number
   scanf("%d", &account);

   // move file pointer to correct record in file
   fseek(fPtr, (account - 1) * sizeof(struct clientData), SEEK_SET);

   // read record from file
   struct clientData client = {0, "", "", 0.0};
   fread(&client, sizeof(struct clientData), 1, fPtr);

   // display error if account does not exist
   if (client.account == 0) {
      printf("Account #%d has no information.\n", account);
   }
   else { // update record
      printf("%-6d%-16s%-11s%10.2f\n\n", client.account, client.lastName,
         client.firstName, client.balance);

      // request transaction amount from user
      printf("%s", "Enter charge (+) or payment (-): ");
      double transaction = 0.0; // transaction amount
      scanf("%lf", &transaction);
      client.balance += transaction; // update record balance

      printf("%-6d%-16s%-11s%10.2f\n", client.account, client.lastName,
         client.firstName, client.balance);

      // move file pointer to correct record in file
      fseek(fPtr, (account - 1) * sizeof(struct clientData), SEEK_SET);

      // write updated record over old record in file
      fwrite(&client, sizeof(struct clientData), 1, fPtr);
   }
}
```

- **Option 1: Create a Formatted List of Accounts**

```
Acct  Last Name       First Name    Balance
29    Brown           Nancy          -24.54
33    Dunn            Stacey         314.33
37    Barker          Doug             0.00
88    Smith           Dave           258.34
96    Stone           Sam             34.98
```

- **Option 2: Update an Account**

```
Enter account to update (1 - 100): 37
37    Barker          Doug              0.00

Enter charge (+) or payment (-): +87.99
37    Barker          Doug             87.99
```

- **Option 3: Create a New Account**

```
Enter new account number (1 - 100): 22
Enter lastname, firstname, balance
? Johnston Sarah 247.45
```

Compiling Multiple-Source-File Programs

- **Multi-File Program Structure**

```
myprogram/
 ├── main.c          # Main program entry point
 ├── utils.c         # Utility functions implementation
 ├── database.c      # Database operations
 ├── main.h          # Main header file
 ├── utils.h         # Utility function prototypes
 └── database.h      # Database function prototypes
```

- **Key Rules**
  - **Function definitions** must be entirely in **one file**
  - **Cannot split** a single function across multiple files
  - **Separate compilation** then linking

Main.c

```
// Main program logic ONLY
#include "main.h"      // Project-wide constants
#include "database.h"  // Database functions
#include "utils.h"     // Utility functions

int main() {
    // Orchestration code
    initialize();
    runApp();
    cleanup();
}
```

Main.h

```
// Constants used everywhere
#define MAX_USERS 100
#define APP_VERSION "1.0"

// Common types
typedef struct {
    int id;
    char name[50];
} User;

// Global configurations
extern const char* DB_PATH;
```

Global Variables Across Files

- **The extern Keyword**
- *// In file1.c*
- int globalCounter = 0; *// Definition*

// In file2.c

extern int globalCounter; *// Declaration (not definition)*

// Can now use globalCounter in file2

- **How It Works**
- File1.c: int flag = 1; ──┐
- │ Linker connects
- File2.c: extern int flag; ──┘
- extern: "This variable exists somewhere else"
- **Linker's job**: Match declarations with definitions
- **Error if**: Linker can't find definition

extern Example

- **Project Structure**

```
// config.c - Variable definition
int debugMode = 1;
float taxRate = 0.075;
// main.c - Using external variables
#include <stdio.h>
extern int debugMode;    // Declare, not define
extern float taxRate;    // Declare, not define
int main() {
    if (debugMode) {        printf("Debug mode ON\n");    }
    printf("Tax rate: %.2f%%\n", taxRate * 100);
    return 0;}
```

Compilation Command

bash

```
gcc main.c config.c -o program
```

Function Prototypes Across Files

- **Extending Function Scope**

```
// In mathutils.c
double calculateAverage(double arr[], int size) {
    // Implementation
}

// In mathutils.h
double calculateAverage(double arr[], int size);  // Prototype

// In main.c
#include "mathutils.h"  // Get prototype
// Can now call calculateAverage()
```

How #include Works

```
#include <stdio.h>
// Copies printf, scanf prototypes into your file
// Linker finds the actual implementations in library
```

The static Keyword (Restricting Scope)

- **static Global Variables**

```
// In file1.c
static int internalCounter = 0;  // Only visible in file1.c
// In file2.c
extern int internalCounter;  // ERROR: Can't access!
// Linker will not find this variable
```

- **static Functions**

```
// In utils.c
static void helperFunction() {  // Internal use only
    // Cannot be called from other files
}
void publicFunction() {  // Can be called from anywhere
    helperFunction();  // OK - same file
}
```